

claude-windows / 75d3d121-f240-4293-afa5-b436a379ad31

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\75d3d121-f240-4293-afa5-b436a379ad31\subagents\workflows\wf_5f13fc79-bad\agent-a71fc21d9ed4131e9.jsonl`  
- SHA-256: `68e45787742a3a315c1d0e64178fa6e6720d80c820102bc254544f3450dc6416`  
- Source modified: `2026-07-08T06:18:57+00:00`  
- Imported at: `2026-07-08T15:59:27+00:00`  
- project: `wf_5f13fc79-bad`  
- session_id: `75d3d121-f240-4293-afa5-b436a379ad31`
```

Transcript

1. user (2026-07-08T06:16:29.937Z)

CONTEXT: This is GitHub issue #521 (Khelsutra/badminton-highlight-indexer), MERGED as PR #525. It makes pipeline phase?machine placement config-driven and dispatches the reel compile/stitch to an L4 Cloud Run worker (NVENC), mirroring the existing detection dispatch.

The full unified diff is at: undefined (read it first).

The full post-change source is under: undefined/ (read whole files there for context ? e.g. undefined/backend/orchestration.py, undefined/backend/worker/__main__.py, undefined/backend/compute/cloud_run.py, undefined/backend/config/placement.py, undefined/backend/config/models.py, undefined/backend/pipeline/stitcher/compiler.py).

Key design invariants that MUST hold (a violation is a real bug):

- DEFAULT-OFF: with no phase_placement config and compute_target unset, behaviour is byte-identical to before #521 (validation+compile on control-plane, detect in-process).
- The cloud compile path must return the SAME {status, filepath, download_url, video_id} shape the SPA already polls for; a stitch failure must be a clean failed-job, never a crash.
- The worker 'compile' subcommand reads a bucket-staged spec (resolved time-pairs + stitching config + source uri) and needs NO control-plane DB. It reuses orchestration._stitch_reel (shared core).
- Encoder: all clips in one reel must share ONE codec so the concat stays -c copy. NVENC is probed once and falls back to libx264 if unavailable (CI/non-GPU) ? the fallback must be all-or-nothing, never per-clip (which would break the concat).
- The config guard rejects validation=cloud_run_l4 without a cloud segment (nobody-validates footgun).
- Marker prefixes: compile uses _compile/, detect uses _dispatch/ ? must not collide.

Only report DEFECTS in the changed code (logic bugs, broken invariants, edge cases that produce a wrong result / crash / silent data loss, or missing test coverage of a real failure mode). Do NOT report style, naming, or speculative concerns. Cite exact file:line from undefined.

A reviewer claims this DEFECT in the merged #521 code. Try HARD to REFUTE it by reading the actual code at undefined. Default to is_real=false if you cannot concretely confirm the failure. Skeptic #1.

CLAIM (high/correctness) backend/orchestration.py:1263

summary: On the L4 worker path, a failed (or skipped) bucket mirror-upload is swallowed and `_stitch_reel` still returns `status='success'`, so a compile job reports success while the reel is never delivered ? the worker's local file is discarded on Job exit and the control-plane has no local copy to fall back to, so the SPA's download 404s (silent data loss reported as success). failure scenario: `phase_placement.compile=cloud_run_l4`. Worker `cmd_compile` -> `_stitch_reel` stitches the reel into scratch, then `storage.put(out_path, reel_uri)` raises (transient GCS 5xx / IAM drift / quota) OR `reel_uri` is empty because the forwarded `storage.output_root` has no `':'` scheme. The except at line 1263 logs a warning and control falls through to ``return {'status':'success', ..., 'reel_uri': reel_uri or ''}``. `cmd_compile` then writes a done-marker with `returncode=0/status=success/download_url` set. `run_compile` reads `rc=0` and `_dispatch_compile_remote` returns `status='success'` with `download_url=/api/highlights/<vid>/<name>`. The user clicks it: `download_highlight()` calls `signed_reel_url` -> `storage.exists(reel_uri)=False` (mirror failed) -> `None`, then falls back to `os.path.join(output_dir, name)` on the CONTROL-PLANE, where the file does not exist (it was stitched on the worker) -> HTTP 404 'No compiled file named ...'. The reel is lost but the job/SPA both show success.

2. user (2026-07-08T06:16:38.996Z)

Found 2 files
backend\worker_main_.py
backend\orchestration.py

3. user (2026-07-08T06:16:40.868Z)

dac6736 perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine placement (#521) (#525)
bf97700 docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells (\$9) (#522)
699d85b feat(validation): cache validation verdict by video_id to skip re-validation on reupload (#518)
71c88ef docs(runlogs): validation-latency closure report for #512 + #513 (cloud-serving) (#519)
115b83d docs(runlogs): record live smoke of the crash-before-marker fast-fail (#515) (#517)
ef41473 fix(cloud-run): fail fast when a worker execution terminates without a done-marker (#515)
f5c9dd9 chore(ui): re-vendor SPA bundle with the hosted compute-picker cloud default (#516)
ec2d080 fix(serving): fail fast when compute=local forces weightless in-process detection (#511)
20862d7 perf(validators): decode scene_cut/blur/relevance frames in one shared pass (#512)
23c1868 fix(serving): worker skips redundant re-validation on the cloud dispatch path (#513)
3be0935 docs(runlog): DEPLOY_REPORT addendum ? control-plane redeployed to the forwarding image (#510)
66756ad chore(serving): forward WASB decode/tuning knobs to worker + release hardening (#509)
143dbb9 feat(serving): default cloud-serving to nvdec_resident decode (#508)
44a2f47 feat(detectors): GPU-resident WASB NVDEC decode behind default-OFF decode_backend flag (#507)
17d844b docs(runlog): alpha-tester "All 1 v 1" cloud-serving run ? timeout diagnosed + downsample fix proven (#504)

4. user (2026-07-08T06:16:45.189Z)

```
1140         if not kept:
1141             raise _CompileError(
1142                 400,
1143                 f"All {len(matched)} matching segments fall below "
1144                 f"stitching.min_shot_count={stitching.min_shot_count}.",
1145             )
1146         return video, kept, out_name
1147
1148
1149     def reel_object_uri(cfg, video_id: str, name: str) -> str:
1150         """The bucket URI a compiled reel is mirrored to for a direct signed-URL download
1151         (#463), or "" when
1152         the output isn't a cloud bucket (truly-local ? serve the local file). Shared by the
1153         compile
```

```

1152         mirror-upload and the /api/highlights redirect:
1153         ``<output_root>/highlights/<video_id>/<name>``. """
1154         root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
1155         if not root or "://" not in root:
1156             return ""
1157         return f"{root.rstrip('/')}/highlights/{video_id}/{name}"
1158
1159     def _storage_settings(cfg) -> Optional[Dict[str, Any]]:
1160         """The active storage settings dict (per-backend config for the storage layer ? S3
1161         endpoint/region,
1162         GCS project/client/signed_url_ttl), or None. Threaded into storage calls so a non-
1163         default backend
1164         isn't silently constructed as a fresh default. """
1165         sc = getattr(cfg, "storage", None)
1166         return sc.model_dump() if sc is not None else None
1167
1168     def _rendition_encode_profile(stitching: Any, rendition: str) -> Optional[Any]:
1169         if rendition != "sd":
1170             return None
1171         renditions = (
1172             stitching.get("renditions")
1173             if isinstance(stitching, dict)
1174             else getattr(stitching, "renditions", None)
1175         )
1176         if isinstance(renditions, dict):
1177             return renditions.get("sd")
1178         return getattr(renditions, "sd", None)
1179
1180     def signed_reel_url(cfg, video_id: str, name: str) -> Optional[str]:
1181         """A short-lived signed URL for a reel mirrored to the output bucket, so
1182         /api/highlights can
1183         307-redirect off the app's response path (bypassing Cloud Run's ~32 MiB cap + the
1184         ephemeral local
1185         /tmp; #463). None ? no bucket copy ? serve the local file. Uses the active storage
1186         config for BOTH
1187         the existence check and the signing (extracted here to keep backend/main.py lean ?
1188         P23/P24). """
1189         reel_uri = reel_object_uri(cfg, video_id, name)
1190         if not reel_uri:
1191             return None
1192         settings = _storage_settings(cfg)
1193         if storage.exists(reel_uri, config=settings):
1194             return storage.signed_url(reel_uri, config=settings, method="GET")
1195         return None
1196
1197     def _compile_wait_message(elapsed_sec: float) -> str:
1198         """The live job.progress line while an L4 compile/stitch is in flight (#521; mirrors
1199         ``_wait_progress_message``). Minute-granular; under a minute reads as cold-start. """
1200         minutes = int(elapsed_sec) // 60
1201         if minutes < 1:
1202             return "stitching on cloud GPU (NVENC) ? starting up"
1203         return f"stitching on cloud GPU (NVENC) ? {minutes} min elapsed"
1204
1205     def _stitch_reel(
1206         cfg,
1207         stitching,
1208         video_id: str,
1209         source_ref: str,
1210         time_pairs: List[Tuple[float, float]],
1211         out_name: str,

```

```

1210     locale: Optional[str],
1211     notify,
1212     rendition: str = "original",
1213 ) -> Dict[str, Any]:
1214     """Stitch + mirror-upload core, shared by the control-plane compile handler AND the
L4 worker
1215     (#521). ``time_pairs`` are already RESOLVED (the caller filtered via
``_resolve_compile``), so this
1216     needs NO control-plane DB ? the worker reads them from the staged spec. Builds the
VideoCompiler
1217     with ``stitching.encoder`` (NVENC on the L4, libx264 on the CP), fetches the source,
runs the
1218     trim/watermark/concat, mirrors the reel to
``<output_root>/highlights/<video_id>/<name>``, and
1219     returns the ``{status, filepath, download_url, video_id, reel_uri}`` the SPA polls
for. Returns a
1220     ``status='failed'`` result on a stitch error (never raises for an expected ffmpeg
failure). The
1221     optional ``rendition`` selects the output encode profile; ``sd`` scales/transcodes
cheaply and
1222     never builds the original-size reel for that job."""
1223     import backend.main as _main # call-time seam: VideoCompiler stays patchable on
backend.main
1224
1225     rendition = rendition or "original"
1226     if rendition not in {"original", "sd"}:
1227         return {
1228             "status": "failed",
1229             "message": f"Unsupported compile rendition: {rendition}",
1230             "video_id": video_id,
1231         }
1232     output_dir = (
1233         getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
1234     )
1235     # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
the job);
1236     # unknown/absent locale keeps the configured default unchanged.
1237     localized_watermark = localize_watermark(stitching.watermark, locale)
1238     encode_profile = _rendition_encode_profile(stitching, rendition)
1239     compiler = _main.VideoCompiler(
1240         output_dir,
1241         begin_padding=stitching.begin_padding,
1242         end_padding=stitching.end_padding,
1243         watermark=localized_watermark,
1244         encoder=getattr(stitching, "encoder", "libx264"),
1245         encode_profile=encode_profile,
1246     )
1247
1248     notify("stitching")
1249     logger.info(
1250         "[compile] stitching video_id=%s rendition=%s output=%s",
1251         video_id,
1252         rendition,
1253         out_name,
1254     )
1255     local_src = ""
1256     try:
1257         # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
path). Resolve to
1258         # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
scratch for a URI (the
1259         # same seam the process path uses); without it, compiling a bucket-ingested
video can't open gs://.
1260         local_src = storage.acquire_local_input(
1261             source_ref, getattr(cfg, "storage", None)

```

```

1262         )
1263         # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count
(for remote
1264         # dispatch) doesn't run multiple compiles at once on the box (see
_LOCAL_COMPUTE_LANE).
1265         with _LOCAL_COMPUTE_LANE:
1266             out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
1267         except Exception as e: # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
FAILURE, not a crash
1268             logger.error(
1269                 "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
1270             )
1271             return {
1272                 "status": "failed",
1273                 "message": f"Highlight stitching failed: {e}",
1274                 "video_id": video_id,
1275             }
1276         finally:
1277             if local_src:
1278                 storage.cleanup_acquired_local_input(
1279                     source_ref, local_src, getattr(cfg, "storage", None)
1280                 )
1281
1282         # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed
URL ? Cloud Run
1283         # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp
copy dies when the
1284         # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch
(the endpoint
1285         # still falls back to the local file on the same instance). On the L4 worker the
mirror is how the
1286         # reel reaches the bucket at all (the worker's local file is discarded when the Job
exits).
1287         reel_uri = reel_object_uri(cfg, video_id, out_name)
1288         if reel_uri:
1289             try:
1290                 notify("uploading reel")
1291                 # Thread the active storage settings so the configured backend (S3
endpoint/region, GCS
1292                 # project/client) is used, not a fresh default.
1293                 storage.put(out_path, reel_uri, config=_storage_settings(cfg))
1294             except Exception as e: # noqa: BLE001 ? best-effort mirror; local stitch
already succeeded
1295                 logger.warning("[compile] could not mirror reel to %s: %s", reel_uri, e)
1296
1297         # `filepath` is server-local (not browser-reachable); also return the download URL
so the SPA needn't
1298         # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
1299         download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
1300         notify("finished")
1301         return {
1302             "status": "success",
1303             "filepath": out_path,
1304             "download_url": download_url,
1305             "video_id": video_id,
1306             "reel_uri": reel_uri or "",
1307             "filename": out_name,
1308             "rendition": rendition,
1309         }
1310
1311
1312     def _dispatch_compile_remote(
1313         cfg,
1314         db,
1315         video_id: str,

```

```

1316     segment_ids: List[int],
1317     output_filename: str,
1318     locale: Optional[str],
1319     notify,
1320     rendition: str = "original",
1321 ) -> Dict[str, Any]:
1322     """#521: run the compile/stitch on the L4 worker instead of the CPU-only control-
plane. The
1323     control-plane owns the DB, so it RESOLVES the reel's segments here
1324     (`_resolve_compile`), stages a
1325     small compile-spec (resolved time-pairs + the resolved stitching/watermark config +
the source URI)
1326     to the bucket, dispatches a ``compile`` Cloud Run Job on the L4 (NVENC), bucket-
polls the
1327     done-marker, and returns the same ``{status, filepath, download_url}`` shape the SPA
expects. The
1328     worker needs NO control-plane DB ? everything it needs rides the spec. Mirrors
1329     ``_maybe_dispatch_remote`` (the DETECT dispatch)."""
1330     from backend.compute import CompileJobSpec, get_compute_backend
1331     notify("resolving")
1332     try:
1333         video, kept, out_name = _resolve_compile(
1334             db, cfg, video_id, segment_ids, output_filename
1335         )
1336     except _CompileError as e:
1337         return {"status": "failed", "message": e.detail, "video_id": video_id}
1338
1339     stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1340     storage_cfg = getattr(cfg, "storage", None)
1341     out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
1342     compute = get_compute_backend(cfg, target_override="cloud_run") if out_root else
None
1343     if compute is None:
1344         # Compile was placed on the L4 but the cloud dispatch isn't actually wired here
1345         # bucket / Cloud Run env). Fall back to the in-process stitch rather than fail ?
1346         # beats no reel. (The placement guard means this is only reachable in a mis-
provisioned setup.)
1347         logger.warning(
1348             "[compile] compile placed on the L4 but no cloud backend available
(out_root=%r) ? "
1349             "stitching in-process on the control-plane instead.",
1350             out_root,
1351         )
1352         time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
1353         return _stitch_reel(
1354             cfg,
1355             stitching,
1356             video_id,
1357             video["filepath"],
1358             time_pairs,
1359             out_name,

```

5. user (2026-07-08T06:17:03.342Z)

```

1     """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3     python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4         [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5     python -m backend.worker train ...           # Gen-0 train-from-bucket (follow-up)
6
7     The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
``local://``

```

```

8      is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
local runs
9      are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10     ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11     the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13     This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14     (validator registry ? segmenter ? report), never a forked pipeline.
15     ""
16
17     from __future__ import annotations
18
19     import argparse
20     import csv
21     import json
22     import logging
23     import os
24     import re
25     import sys
26     import tempfile
27     from typing import Any, List
28
29     from backend import storage
30     from backend.compute import ComputeJobSpec, get_compute_backend
31     from backend.config import (
32         StartupCheckError,
33         enforce_startup_checks,
34         load_config,
35         probe_cuda_available,
36     )
37     from backend.infrastructure.database import Database
38     from backend.pipeline.segmenters.base import segmenter_registry
39     from backend.pipeline.validators.base import registry as validator_registry
40     from backend.reporting import emit_indexer_report
41     from backend.results import Failure
42     from backend.utils.ffmpeg import ffprobe_bin # P2a: single ffmpeg/ffprobe resolver
43     from backend.utils.hashing import compute_video_id
44
45     logger = logging.getLogger("backend.worker")
46
47
48     def _coerce(v: str) -> Any:
49         """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
str."""
50         low = v.lower()
51         if low in ("true", "false"):
52             return low == "true"
53         for cast in (int, float):
54             try:
55                 return cast(v)
56             except ValueError:
57                 pass
58         return v
59
60
61     def _apply_overrides(config: Any, sets: List[str]) -> None:
62         """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
63         for kv in sets or []:
64             key, sep, val = kv.partition("=")
65             if not sep:
66                 raise ValueError(f"--set expects k=v, got {kv!r}")
67             parts = key.split(".")

```

```

68         obj = config
69         for p in parts[:-1]:
70             obj = getattr(obj, p)
71             setattr(obj, parts[-1], _coerce(val))
72
73
74     def _write_intervals_csv(segments: List[dict], path: str) -> None:
75         """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
76         artifact (same schema as the rally-annotator labels)."""
77         with open(path, "w", newline="") as f:
78             w = csv.writer(f)
79             w.writerow(["start", "end", "shot_count", "ending_reason"])
80             for s in segments:
81                 w.writerow(
82                     [
83                         f"{float(s.get('start_time', 0.0)):.3f}",
84                         f"{float(s.get('end_time', 0.0)):.3f}",
85                         s.get("shot_count", ""),
86                         s.get("ending_reason", s.get("shot_count_confidence", "")),
87                     ]
88                 )
89
90
91     def _write_report_json(
92         video_id: str, segments: List[dict], status: str, path: str, video_uri: str = ""
93     ) -> None:
94         with open(path, "w") as f:
95             json.dump(
96                 {
97                     "video_id": video_id,
98                     "status": status,
99                     "segment_count": len(segments),
100                     # The SOURCE the detection ran on (CP-rebuild P3, #485): a restart-wiped
101                     # control plane re-hydrating from outputs/<md5>/ restores the video row
102                     # with its truthful path ? without this, only the segments recover.
103                     "video_uri": video_uri,
104                     "segments": [
105                         {
106                             "start": float(s.get("start_time", 0.0)),
107                             "end": float(s.get("end_time", 0.0)),
108                         }
109                         for s in segments
110                     ],
111                 },
112                 f,
113                 indent=2,
114             )
115
116
117     def _serialize_and_push_outputs(
118         scratch: str,
119         out_uri: str,
120         video_id: str,
121         segments: List[dict],
122         status: str,
123         storage_cfg: dict,
124         video_uri: str = "",
125     ) -> List[str]:
126         """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and
push both to
127         ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.
128
129         Shared by the SUCCESS path and the failure path (`_fail`) so a detector
timeout/abort leaves a

```



```

130     ``status="failed"`` report.json in the bucket ? the control plane's source of truth
131     (`orchestration.cached_process_result` / ``probe_process_cache`` treat any
non-``success``
132     report as "no usable result" ? retry, and the restart-recovery poll waits for
report.json to
133     EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``. ""
134     out_local = os.path.join(scratch, "outputs", video_id)
135     os.makedirs(out_local, exist_ok=True)
136     _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
137     _write_report_json(
138         video_id,
139         segments,
140         status,
141         os.path.join(out_local, "report.json"),
142         video_uri=video_uri,
143     )
144     out_prefix = out_uri.rstrip("/")
145     pushed: List[str] = []
146     for fn in sorted(os.listdir(out_local)):
147         uri = f"{out_prefix}/outputs/{video_id}/{fn}"
148         storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
149         pushed.append(uri)
150     return pushed
151
152
153     def _run_startup_checks(config: Any) -> bool:
154         """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
155         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
156         cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
157         (returns True, ZERO work) when no deployment.profile is selected.
158
159         The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
160         torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
161         WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
162         no-op of work, not just of output."""
163         profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
164         cuda = (
165             probe_cuda_available() if profile else None
166         ) # torch-free on the default-OFF path
167         try:
168             enforce_startup_checks(config, cuda_available=cuda, logger=logger)
169             return True
170         except StartupCheckError:
171             return False # already logged at ERROR by enforce_startup_checks
172
173
174     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
175         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
176         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
177         container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.
178
179         A remote job that never writes a completion marker exits via one of two CLEAN codes
(distinct
180         from the local 2/3), never a raw traceback:
181         * rc 4 ? the bucket poll exhausted its budget (``PolicyTimeout``): the execution
may still be

```

```

182         running (the shim best-effort cancels it).
183     * rc 5 ? the execution has TERMINATED without a marker
184     (`RemoteExecutionFailed`): a worker
185         crash/OOM/pre-marker abort caught FAST by the Executions-API probe, so instead
of the
186         -65-min opaque wait the operator gets an immediate, log-pointing failure.""
187     from backend.compute import RemoteExecutionFailed
188     from backend.infrastructure.execution_policy import PolicyTimeout
189
190     spec = ComputeJobSpec(
191         video_uri=args.video_uri,
192         out_uri=args.out_uri,
193         segmenter=args.segmenter,
194         config_overrides=tuple(args.set or ()),
195         skip_validation=bool(getattr(args, "skip_validation", False)),
196     )
197     try:
198         result = compute.run_infer(spec)
199     except RemoteExecutionFailed as e:
200         logger.error(
201             "[worker] remote execution terminated without a completion marker: %s", e
202         )
203         return 5
204     except PolicyTimeout as e:
205         logger.warning(
206             "[worker] remote compute did not complete (no marker within budget): %s", e
207         )
208         return 4
209     logger.info(
210         "[worker] remote compute done: video_id=%s rc=%d outputs=%s",
211         result.video_id,
212         result.returncode,
213         result.outputs_uri,
214     )
215     return result.returncode
216
217     def _write_done_marker(
218         done_uri: str,
219         video_id: str,
220         returncode: int,
221         segment_count: int,
222         status: str,
223         storage_cfg: dict,
224         scratch: str,
225     ) -> None:
226         """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
227         dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
228     try:
229         marker = {
230             "video_id": video_id,
231             "returncode": returncode,
232             "segment_count": segment_count,
233             "status": status,
234         }
235         local = os.path.join(scratch, "_done.json")
236         with open(local, "w", encoding="utf-8") as f:
237             json.dump(marker, f)
238         storage.put(local, done_uri, config=storage_cfg)
239         logger.info("[worker] wrote completion marker -> %s", done_uri)
240     except Exception as e: # never fail a good run because the marker push hiccuped
241         logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
242

```

```

243
244     def cmd_infer(args: argparse.Namespace) -> int:
245         config = load_config(args.config) if args.config else load_config()
246         _apply_overrides(config, args.set)
247         if not _run_startup_checks(config):
248             return 2
249
250         # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
251         # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
252         # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
253         # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
254         compute = get_compute_backend(config)
255         if compute is not None:
256             return _dispatch_remote(compute, args)
257
258         storage_cfg = config.storage.model_dump()
259
260         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
261         os.makedirs(scratch, exist_ok=True)
262         config.output.output_dir = scratch
263
264         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
265         local_video = storage.fetch(
266             args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg
267         )
268         print(f"[worker] pulled {args.video_uri} -> {local_video}")
269
270         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
271         video_id = compute_video_id(local_video)
272
273         # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
274         # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate:
it runs the
275         # SAME validators with ITS resolved config and only dispatches on PASS. Re-
validating here is
276         # redundant, decodes the whole video a SECOND time, and is outright WRONG when the
worker's config
277         # resolves different thresholds than the control-plane's ? observed 2026-07-07: a
5312x2988 clip
278         # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0,
then the worker
279         # REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the
fact.
280         # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote)
skips it; the
281         # direct ``worker infer`` CLI has no control-plane, so it still validates by
default.
282         if getattr(args, "skip_validation", False):
283             print(
284                 "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
285                 "gate and already validated this input."
286             )
287             val_results, val_context = [], {}
288         else:
289             val_results, val_context = validator_registry.run_all_with_context(
290                 local_video, config
291             )
292         failures = [r for r in val_results if not r.passed]

```

```

293     db = Database(os.path.join(scratch, config.output.db_name))
294     db.add_video(
295         video_id,
296         local_video,
297         "failed" if failures else "processed",
298         [r.model_dump() for r in val_results],
299     )
300
301     def _fail(rc: int) -> int:
302         # A worker-side FAILURE (validation, unknown segmenter, or a detector
303         # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
304         # a status="failed" report.json/intervals.csv (0 segments) so the control
305         # truth (submit-cache probe, restart-recovery) and the alpha-user result surface
306         # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
307         # mask the original failure. THEN the M6 done-marker (real rc + status) so a
308         # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
309         # expires.
310         try:
311             _serialize_and_push_outputs(
312                 scratch,
313                 args.out_uri,
314                 video_id,
315                 [],
316                 "failed",
317                 storage_cfg,
318                 str(getattr(args, "video_uri", "") or ""),
319             )
320         except Exception as e: # never mask the real failure on an artifact-push hiccup
321             logger.warning("[worker] could not push failure artifacts: %s", e)
322         # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
323         if getattr(args, "done_uri", None):
324             _write_done_marker(
325                 args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
326             )
327         return rc
328
329     segments: List[dict] = []
330     segmenter_actual = None
331     segmentation_ran = False
332     status = "failed"
333     try:
334         if failures:
335             print(
336                 "[worker] validation FAILED: "
337                 + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
338             )
339             return _fail(2)
340         seg_name = args.segmenter or config.indexing.default_segmenter
341         segmenter_cls = segmenter_registry.get(seg_name)
342         if not segmenter_cls:
343             print(
344                 f"[worker] unknown segmenter: {seg_name!r} "
345                 f"(have {list(segmenter_registry.list_available())})"
346             )
347             return _fail(2)
348         segmenter_actual = seg_name
349         result = segmenter_cls(db, config).process_video(
350             video_id, local_video, max_frames=args.max_frames
351         )

```

```

351         segmentation_ran = True
352     if isinstance(result, Failure):
353         print(f"[worker] segmenter FAILED: {result.message}")
354         return _fail(3)
355     segments = result.value if hasattr(result, "value") else result
356     status = "success"
357     # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
358     # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
359     # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
360     if not segments:
361         detector_impl = (
362             os.environ.get("RALLY_DETECTOR_IMPL")
363             or getattr(config.indexing, "detector_impl", None)
364             or "auto"
365         )
366         logger.warning(
367             "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
368             "The detector substrate yielded no usable trajectories/windows ? check
the detector "
369             "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "
370             "skip_ai_handoff=False (the segmenter bails BEFORE detection ? set the
key or "
371             "indexing.skip_ai_handoff=true; cloud-serving now fails this at
startup), native "
372             "runner UNAVAILABLE (weights/repo/WASB_PYTHON env), the WASB env
produced no "
373             "detections, or the input resolution differs from the tuned proxy
resolution. "
374             "Re-run with -v for the native command + frame counts.",
375             video_id,
376             segmenter_actual,
377             detector_impl,
378         )
379     finally:
380         # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
381         emit_indexer_report(
382             db=db,
383             video_id=video_id,
384             video_path=local_video,
385             config=config,
386             val_results=val_results,
387             val_context=val_context,
388             segmenter_requested=args.segmenter,
389             segmenter_actual=segmenter_actual,
390             was_reingest=False,
391             segmentation_ran=segmentation_ran,
392         )
393
394     # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
395     pushed = _serialize_and_push_outputs(
396         scratch,
397         args.out_uri,
398         video_id,
399         segments,
400         status,
401         storage_cfg,
402         str(getattr(args, "video_uri", "") or ""),
403     )
404
405     print(

```

```

406         f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):"
407     )
408     for u in pushed:
409         print("    ", u)
410
411     # M6: a remote dispatcher passes --done-uri; write the SUCCESS completion marker so
its bucket-poll
412     # terminates (carries video_id ? the dispatcher's output path + rc=0). Worker
FAILURES (rc 2/3)
413     # write their own marker via _fail() above, so the dispatcher fails FAST either way
(no 30-min hang
414     # on a bad video). DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
415     if getattr(args, "done_uri", None):
416         _write_done_marker(
417             args.done_uri, video_id, 0, len(segments), status, storage_cfg, scratch
418         )
419     return 0
420
421
422     def _read_compile_spec(spec_uri: str, storage_cfg: dict, scratch: str) -> dict:
423         """Fetch + parse the control-plane-staged compile-spec (#521): the resolved
`time_pairs`, the
424         resolved `stitching` config, the source `video_uri`, `video_id`,
`output_name` + `locale`.
425         A gs://|s3:// URI is downloaded to scratch first (cloud backends require a real
dst)."""
426         local = storage.fetch(
427             spec_uri, os.path.join(scratch, "compile_spec.json"), config=storage_cfg
428         )
429         with open(local, encoding="utf-8") as f:
430             data = json.load(f)
431         if not isinstance(data, dict):
432             raise ValueError(f"compile spec at {spec_uri} is not a JSON object")
433         return data
434
435
436     def _write_compile_marker(
437         done_uri: str,
438         result: dict,
439         video_id: str,
440         returncode: int,
441         storage_cfg: dict,
442         scratch: str,
443     ) -> None:
444         """#521: write the compile done-marker (rc + status + the reel's
download_url/reel_uri) so the
445         dispatcher's bucket-poll terminates and can return the SPA result. Best-effort ?
never raises."""
446         try:
447             marker = {
448                 "video_id": video_id,
449                 "returncode": returncode,
450                 "status": result.get("status", "failed"),
451                 "download_url": result.get("download_url", ""),
452                 "reel_uri": result.get("reel_uri", ""),
453                 "filename": result.get("filename", ""),
454                 "rendition": result.get("rendition", ""),
455                 "message": result.get("message", ""),
456             }
457             local = os.path.join(scratch, "_compile_done.json")
458             with open(local, "w", encoding="utf-8") as f:
459                 json.dump(marker, f)
460             storage.put(local, done_uri, config=storage_cfg)

```

```

461         logger.info("[worker] wrote compile marker -> %s", done_uri)
462     except Exception as e: # never fail a good stitch because the marker push hiccuped
463         logger.warning("[worker] could not write compile done-marker %s: %s", done_uri,
464 e)
465
466     def cmd_compile(args: argparse.Namespace) -> int:
467         """#521: run the reel COMPILE/stitch on THIS worker (the L4 has NVENC + 8 vCPU). The
468 stitch is the
469         2026-07-08 CUJ's dominant term (~13 min on the 2-vCPU control-plane for an 11-clip
470 4K reel). Read
471         the control-plane-staged compile-spec (resolved time-pairs + stitching config +
472 source URI), stitch
473         the reel via the SAME ``orchestration._stitch_reel`` core the control-plane uses (so
474 there is no
475         forked stitch pipeline), upload the reel to the bucket, and write a done-marker for
476 the dispatcher's
477         bucket-poll. No control-plane DB is needed ? the spec carries everything the stitch
478 requires."""
479         from backend import orchestration
480         from backend.config import StitchingConfig
481
482         config = load_config(args.config) if args.config else load_config()
483         _apply_overrides(config, args.set)
484         storage_cfg = config.storage.model_dump()
485
486         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-compile-")
487         os.makedirs(scratch, exist_ok=True)
488         # Clips + the assembled reel land in scratch; the reel is then mirror-uploaded to
489 the bucket.
490         config.output.output_dir = scratch
491         # The dispatcher forwards storage.output_root via --set; --out-uri carries the same
492 value as a
493         # fallback so reel_object_uri (which reads storage.output_root) always resolves the
494 reel's bucket
495         # destination even if the override were dropped.
496         if not (config.storage.output_root or "").strip():
497             config.storage.output_root = args.out_uri
498
499         spec = _read_compile_spec(args.spec_uri, storage_cfg, scratch)
500         video_id = str(spec.get("video_id") or "")
501         out_name = str(spec.get("output_name") or "")
502         rendition = str(spec.get("rendition") or "original")
503         source_ref = str(spec.get("video_uri") or "")
504         locale = spec.get("locale")
505         time_pairs = [
506             (float(pair[0]), float(pair[1])) for pair in (spec.get("time_pairs") or [])
507         ]
508         stitching = StitchingConfig(**(spec.get("stitching") or {}))
509
510         print(
511             f"[worker] compile: video_id={video_id} clips={len(time_pairs)} "
512             f"encoder={stitching.encoder} rendition={rendition} out={out_name}"
513         )
514
515         result = orchestration._stitch_reel(
516             config,
517             stitching,
518             video_id,
519             source_ref,
520             time_pairs,
521             out_name,
522             locale,
523             lambda stage: print(f"[worker] compile: {stage}"),
524             rendition,

```

```

516     )
517     status = str(result.get("status") or "failed")
518     rc = 0 if status == "success" else 3
519     if status != "success":
520         print(f"[worker] compile FAILED: {result.get('message')}")
521
522     # DEFAULT-OFF: no --done-uri ? no marker (a direct-CLI stitch). A remote dispatcher
passes it.
523     if getattr(args, "done_uri", None):
524         _write_compile_marker(args.done_uri, result, video_id, rc, storage_cfg, scratch)
525     return rc
526
527
528     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
529     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
530     _VIDEO_EXTS = ("mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
531
532     #: Leading ISO-date prefix on a canonical label name (`labels/<YYYY-MM-
DD><name>.rallies.csv`,
533     #: docs/DATA_IN_GCS.md §7.1). It disambiguates recycled GoPro names by authoring date
but is NOT
534     #: part of the clip's join key, so it's stripped when deriving the videos/ stem.
535     _LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")
536
537
538     def _label_clip_name(fname: str) -> str:
539         """Derive the clip name (the `videos/<name>.<ext>` join key) from a label
filename.
540
541         Strips both the `rallies.csv` / `csv` suffix and an optional leading ISO-date
prefix, so a
542         canonical dated label and a legacy bare label resolve to the SAME bare clip stem:
543
544         * `2026-06-22_GX020094.rallies.csv` -> `GX020094` (canonical, date-
stamped)
545         * `GX020094.rallies.csv` -> `GX020094` (legacy bare ? same
stem)
546         * `GX020094_proxy.rallies.csv` -> `GX020094_proxy` (`_proxy` is part of
the name, kept)
547         * `2026-06-21_mahadevpura_1.rallies.csv` -> `mahadevpura_1` (underscores in
<name> survive)
548
549         Without the date strip, `--trajectory-source detector` can't match a dated label
to its bare
550         video stem, so every clip is skipped and train aborts with `<2 videos`
(DATA_IN_GCS §7b #1).
551         """
552         name = fname.replace(".rallies.csv", "").replace(".csv", "")
553         return _LABEL_DATE_PREFIX.sub("", name)
554
555
556     def _probe_video_fps_width(path: str):
557         """Robust ffmpeg (fps, frame_width) for a real video. Returns None on ANY failure.
558
559         The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
560         fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
561         cv2's `_probe_fps_width` silently defaults to (30, 1280) on a read miss ? fine for
the
562         inference report (it flags the fallback) but corrupting for REAL training data. So
here we
563         probe with ffmpeg (same tool as `get_video_duration`) and return None so the

```



```

caller SKIPS
564         rather than guesses.
565         """
566         import json
567         import subprocess
568
569         try:
570             out = subprocess.check_output(
571                 [
572                     ffprobe_bin(),
573                     "-v",
574                     "error",
575                     "-select_streams",
576                     "v:0",
577                     "-show_entries",
578                     "stream=r_frame_rate,width",
579                     "-of",
580                     "json",
581                     path,
582                 ],
583                 stderr=subprocess.DEVNULL,
584             ).decode()
585             st = (json.loads(out).get("streams") or [{}])[0]
586             num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
587             fps = float(num) / float(den or "1")
588             width = float(st.get("width") or 0)
589             if fps > 0 and width > 0:
590                 return fps, width
591         except (
592             subprocess.SubprocessError,
593             ValueError,
594             KeyError,
595             OSError,
596             json.JSONDecodeError,
597         ):
598             pass
599         return None
600
601
602     def _resolve_train_detector(args: argparse.Namespace, config: Any):
603         """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
604         detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
605         box,
606         stub=CPU mock). Returns the runner, or prints an error + returns None.
607
608         Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
609         the
610         detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
611         has
612         no shuttle detector and is rejected.
613         """
614         from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
615
616         seg_cls = segmenter_registry.get(args.segmenter)
617         if not seg_cls:
618             print(
619                 f"[worker] unknown segmenter: {args.segmenter!r} "
620                 f"(have {list(segmenter_registry.list_available())})"
621             )
622             return None
623         seg = seg_cls(None, config)
624         if not isinstance(seg, TrajectoryHybridSegmenter):
625             print(
626                 f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
627                 f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid / fusion_hybrid)."
```

```

625         )
626         return None
627     try:
628         runner, _ = seg._resolve_runner(config.indexing)
629     except NotImplementedError as e:
630         # e.g. detector_impl='native' for a family without a native runner (tracknet, or
631         # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
632         traceback.
633         print(f"[worker] detector not available: {e}")
634         return None
635     ok, msg = runner.healthcheck()
636     if not ok:
637         print(f"[worker] detector environment not ready: {msg}")
638         return None
639     print(f"[worker] detector ready ({args.segmenter}): {msg}")
640     return runner
641
642 def cmd_train(args: argparse.Namespace) -> int:
643     """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
644     shell out
645     to training.gen0.harness (backend must NOT import training) -> push the artifact
646     bundle.
647     Two trajectory sources (the train pipeline + harness are identical either way):
648     * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
649     synthetic
650     shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
651     CI.
652     * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
653     DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
654     This
655     is the M3.5 "first real training" path; only the trajectory source flips.
656     """
657     import subprocess
658
659     config = load_config(args.config) if args.config else load_config()
660     _apply_overrides(config, args.set)
661     if not _run_startup_checks(config):
662         return 2
663     storage_cfg = config.storage.model_dump()
664
665     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
666     labels_dir = os.path.join(scratch, "labels")
667     traj_dir = os.path.join(scratch, "trajectories")
668     videos_dir = os.path.join(scratch, "videos")
669     os.makedirs(labels_dir, exist_ok=True)
670     os.makedirs(traj_dir, exist_ok=True)
671
672     corpus = args.corpus_uri.rstrip("/")
673     label_uris = sorted(
674         u
675         for u in storage.list_uris(f"{corpus}/labels/", config=storage_cfg)
676         if u.lower().endswith(".csv")
677     )
678     if len(label_uris) < 2:
679         print(
680             f"[worker] need >=2 labeled videos for leave-one-video-out; found
681             {len(label_uris)} "
682             f"under {corpus}/labels/"
683         )
684         return 2
685
686     # Real-detector source: resolve the runner once + index the bucket's videos/ by
687     stem.

```

```

682 runner = None
683 video_by_stem: dict = {}
684 if args.trajectory_source == "detector":
685     os.makedirs(videos_dir, exist_ok=True)
686     runner = _resolve_train_detector(args, config)
687     if runner is None:
688         return 2
689     for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
690         vname = storage.parse_uri(vuri).name
691         if not vname.lower().endswith(_VIDEO_EXTS):
692             continue # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
693         video_by_stem[os.path.splitext(vname)[0]] = vuri
694
695 print(f"[worker] trajectory source: {args.trajectory_source}")
696 specs: List[dict] = []
697 for uri in label_uris:
698     fname = storage.parse_uri(uri).name # e.g. 2026-06-22_GX020094.rallies.csv
699     name = _label_clip_name(fname) # -> GX020094 (date prefix stripped)
700     local_label = storage.fetch(
701         uri, os.path.join(labels_dir, fname), config=storage_cfg
702     )
703     if args.trajectory_source == "detector":
704         vuri = video_by_stem.get(name)
705         if not vuri:
706             print(
707                 f"[worker] no video for {name!r} under {corpus}/videos/ ? skipping."
708             )
709             continue
710         local_video = storage.fetch(
711             vuri,
712             os.path.join(videos_dir, storage.parse_uri(vuri).name),
713             config=storage_cfg,
714         )
715         video_id = compute_video_id(local_video)
716         # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
717         # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
718         meta = _probe_video_fps_width(local_video)
719         if meta is None:
720             print(
721                 f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess)."
722             )
723             continue
724         fps, frame_width = meta
725         traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
726         if not traj:
727             print(
728                 f"[worker] detector produced no trajectory for {name!r} ? skipping."
729             )
730             continue
731         specs.append(
732             {
733                 "name": name,
734                 "trajectory": traj,
735                 "labels": local_label,
736                 "fps": fps,
737                 "frame_width": frame_width,
738             }
739         )
740     else:
741         from backend.pipeline.detectors.stub_runner import (
742             synth_trajectory_from_labels,

```

```

743         )
744
745         traj = os.path.join(traj_dir, f"{name}_traj.csv")
746         synth_trajectory_from_labels(
747             local_label, traj, fps=args.fps, frame_width=args.frame_width
748         )
749         specs.append(
750             {
751                 "name": name,
752                 "trajectory": traj,
753                 "labels": local_label,
754                 "fps": args.fps,
755                 "frame_width": args.frame_width,
756             }
757         )
758
759     if len(specs) < 2:
760         print(
761             f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}."
762         )
763         return 2
764     manifest_path = os.path.join(scratch, "manifest.json")
765     with open(manifest_path, "w", encoding="utf-8") as f:
766         json.dump(specs, f, indent=2)
767     src_label = (
768         "real detector" if args.trajectory_source == "detector" else "CPU-mock stub"
769     )
770     print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")
771
772     out_dir = os.path.join(scratch, "artifacts")
773     cmd = [
774         sys.executable,
775         "-m",
776         "training.gen0.harness",
777         "--manifest",
778         manifest_path,
779         "--out",
780         out_dir,
781         "--version",
782         args.version,
783     ]
784     if args.floor:
785         floor_local = (
786             storage.fetch(
787                 args.floor, os.path.join(scratch, "floor.json"), config=storage_cfg
788             )
789             if "://" in args.floor
790             else args.floor
791         )
792         cmd += ["--floor", floor_local]
793     print("[worker] training:", " ".join(cmd))
794     rc = subprocess.run(cmd).returncode
795     if rc != 0:
796         print(f"[worker] gen0 harness failed (rc={rc})")
797         return rc
798
799     # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
800     bundle = os.path.join(out_dir, args.version)
801     out_prefix = args.out_uri.rstrip("/")
802     pushed = []
803     for fn in sorted(os.listdir(bundle)):
804         uri = f"{out_prefix}/weights/{args.version}/{fn}"
805         storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
806         pushed.append(uri)

```

```

807     print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
808     for u in pushed:
809         print("    ", u)
810     return 0
811
812
813 def build_parser() -> argparse.ArgumentParser:
814     p = argparse.ArgumentParser(
815         prog="python -m backend.worker",
816         description="Storage-aware worker: pull ? run pipeline ? push.",
817     )
818     p.add_argument(
819         "-v",
820         "--verbose",
821         action="store_true",
822         help="DEBUG logging (the native detector command, frame counts, per-stage
detail)",
823     )
824     sub = p.add_subparsers(dest="cmd", required=True)
825
826     pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
827     pi.add_argument(
828         "--video-uri", required=True, help="local path / local:// / s3:// / gcs://"
829     )
830     pi.add_argument(
831         "--out-uri",
832         required=True,
833         help="output prefix (outputs/<video_id>/? is appended)",
834     )
835     pi.add_argument(
836         "--segmenter", default=None, help="default: indexing.default_segmenter"
837     )
838     pi.add_argument(
839         "--config", default=None, help="config.json path (default: ./config.json)"
840     )
841     pi.add_argument(
842         "--scratch", default=None, help="local scratch dir (default: a temp dir)"
843     )
844     pi.add_argument("--max-frames", type=int, default=None)
845     pi.add_argument(
846         "--done-uri",
847         default=None,
848         help="M6: storage URI for a completion MARKER (video_id + rc) written on a "
849         "successful run, so a remote (Cloud Run) dispatcher's bucket-poll terminates. "
850         "Default-OFF: omit it for the normal local run.",
851     )
852     pi.add_argument(
853         "--set",
854         action="append",
855         default=[],
856         metavar="k=v",
857         help="config override, e.g. --set indexing.skip_ai_handoff=true",
858     )
859     pi.add_argument(
860         "--skip-validation",
861         action="store_true",
862         help="skip the worker's re-validation: the dispatcher (control-plane) already
ran the "
863         "validators with its resolved config and only dispatches on PASS, so re-
validating here is "
864         "redundant, decodes the video again, and can contradict the gate. Default-OFF:
the direct "
865         "`worker infer` CLI still validates.",
866     )
867     pi.set_defaults(func=cmd_infer)

```

```

868
869     pc = sub.add_parser(
870         "compile",
871         help="#521: stitch a highlight reel on this worker (L4 NVENC) from a staged
compile-spec",
872     )
873     pc.add_argument(
874         "--spec-uri",
875         required=True,
876         help="storage URI of the control-plane-staged compile-spec JSON (time-pairs +
stitching "
877             "config + source uri + video_id)",
878     )
879     pc.add_argument(
880         "--out-uri",
881         default="",
882         help="output ROOT (gs://? bucket); the reel is mirrored to
highlights/<video_id>/<name>. "
883             "Also forwarded via --set storage.output_root; either resolves the reel path.",
884     )
885     pc.add_argument(
886         "--done-uri",
887         default=None,
888         help="storage URI for the compile completion MARKER (rc + download_url), so a
remote "
889             "dispatcher's bucket-poll terminates. Default-OFF: omit for a direct-CLI
stitch.",
890     )
891     pc.add_argument(
892         "--config", default=None, help="config.json path (default: ./config.json)"
893     )
894     pc.add_argument(
895         "--scratch", default=None, help="local scratch dir (default: a temp dir)"
896     )
897     pc.add_argument(
898         "--set",
899         action="append",
900         default=[],
901         metavar="k=v",
902         help="config override, e.g. --set storage.output_root=gs://bucket",
903     )
904     pc.set_defaults(func=cmd_compile)
905
906     pt = sub.add_parser(
907         "train",
908         help="Gen-0 train-from-bucket (labels [+videos] -> trajectories -> harness)",
909     )
910     pt.add_argument(
911         "--corpus-uri",
912         required=True,
913         help="prefix containing labels/ (+ videos/ for "
914             "--trajectory-source detector), e.g. s3://bucket or gcs://bucket",
915     )
916     pt.add_argument(
917         "--out-uri",
918         required=True,
919         help="output prefix (weights/<version>/ is appended)",
920     )
921     pt.add_argument("--version", required=True, help="semver for the artifact bundle")
922     pt.add_argument(
923         "--trajectory-source",
924         choices=["synth", "detector"],
925         default="synth",
926         help="synth = CPU-mock label-consistent trajectories (default, no GPU); "
927             "detector = REAL shuttle trajectories via the resolved DetectorRunner "

```

```

928         "(needs videos/ in the corpus; honours detector_impl).",
929     )
930     pt.add_argument(
931         "--segmenter",
932         default="wasb_hybrid",
933         help="trajectory hybrid whose detector is used for --trajectory-source
detector",
934     )
935     pt.add_argument(
936         "--floor", default=None, help="floor baseline JSON (local path or storage URI)"
937     )
938     pt.add_argument("--config", default=None)
939     pt.add_argument("--scratch", default=None)
940     pt.add_argument(
941         "--fps",
942         type=float,
943         default=30.0,
944         help="synth trajectory fps (detector probes the video)",
945     )
946     pt.add_argument(
947         "--frame-width",
948         type=float,
949         default=1280.0,
950         help="synth trajectory frame width (detector probes the video)",
951     )
952     pt.add_argument("--set", action="append", default=[], metavar="k=v")
953     pt.set_defaults(func=cmd_train)
954     return p
955
956
957     def _setup_logging(verbose: bool) -> None:
958         """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
diagnostics are
959         VISIBLE ? the worker previously ran with no logging config, so every
segmenter/detector log
960         (detector choice, 'Running native WASB inference', availability failures) was
silently dropped.
961         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
(the CLI
962         entry), not at import, so importing the module never reconfigures a caller's
logging."""
963         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
library
964         # configured logging at import), which would leave the level at WARNING and re-
suppress the
965         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
verbosity.
966         logging.basicConfig(
967             level=logging.DEBUG if verbose else logging.INFO,
968             format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
969             datefmt="%H:%M:%S",
970             force=True,
971         )
972
973
974     def main(argv=None) -> int:
975         args = build_parser().parse_args(argv)
976         _setup_logging(getattr(args, "verbose", False))
977         return args.func(args)
978
979
980     if __name__ == "__main__":
981         sys.exit(main())
982

```

6. user (2026-07-08T06:17:28.809Z)

```
1360         locale,
1361         notify,
1362         rendition,
1363     )
1364
1365     # Build the compile-spec the worker consumes. time-pairs are JSON lists (frozen-safe
on the wire);
1366     # the stitching config is the control-plane's RESOLVED one, with the encoder swapped
to the L4's
1367     # (deployment.cloud_stitch_encoder, NVENC by default) so the worker hardware-encodes
the reel.
1368     stitch_dump = stitching.model_dump()
1369     stitch_dump["encoder"] = getattr(
1370         getattr(cfg, "deployment", None), "cloud_stitch_encoder", "h264_nvenc"
1371     )
1372     spec_payload = {
1373         "video_id": video_id,
1374         "video_uri": video["filepath"],
1375         "output_name": out_name,
1376         "rendition": rendition,
1377         "locale": locale,
1378         "time_pairs": [
1379             [float(s["start_time"]), float(s["end_time"])] for s in kept
1380         ],
1381         "stitching": stitch_dump,
1382     }
1383     token = uuid.uuid4().hex
1384     spec_uri = f"{out_root.rstrip('/')}/_compile/{token}/spec.json"
1385     try:
1386         notify("staging compile spec")
1387         with tempfile.TemporaryDirectory(prefix="cr-compile-") as td:
1388             local_spec = os.path.join(td, "spec.json")
1389             with open(local_spec, "w", encoding="utf-8") as f:
1390                 json.dump(spec_payload, f)
1391             storage.put(local_spec, spec_uri, config=_storage_settings(cfg))
1392     except Exception as e: # noqa: BLE001 ? a staging failure is a clean job failure,
not a crash
1393         logger.error(
1394             "[compile] failed to stage the compile spec to %s: %s", spec_uri, e,
exc_info=True
1395         )
1396     return {
1397         "status": "failed",
1398         "message": f"cloud compile: could not stage the compile spec: {e}",
1399         "video_id": video_id,
1400     }
1401
1402     # Forward the bucket coordinates so the worker (no DEPLOYMENT_PROFILE) can read the
spec, mirror the
1403     # reel, and write the marker ? mirrors how _maybe_dispatch_remote forwards the WASB
knobs for detect.
1404     overrides = [f"storage.output_root={out_root}"]
1405     backend_name = (getattr(storage_cfg, "backend", "") or "").strip()
1406     if backend_name:
1407         overrides.append(f"storage.backend={backend_name}")
1408     spec = CompileJobSpec(
1409         out_uri=out_root,
1410         spec_uri=spec_uri,
1411         video_id=video_id,
1412         output_name=out_name,
1413         config_overrides=tuple(overrides),
1414     )
1415
```



```

1416         notify("dispatching stitch (cloud_run)")
1417         logger.info(
1418             "[compile] dispatching stitch for video_id=%s rendition=%s -> Cloud Run Job
(out=%s, %d clip(s))",
1419             video_id,
1420             rendition,
1421             out_root,
1422             len(kept),
1423         )
1424         try:
1425             result = compute.run_compile(
1426                 spec, on_wait=lambda elapsed: notify(_compile_wait_message(elapsed))
1427             )
1428         except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
1429             logger.error("[compile] cloud compile dispatch error: %s", e, exc_info=True)
1430             return {
1431                 "status": "failed",
1432                 "message": f"cloud compile dispatch error: {e}",
1433                 "video_id": video_id,
1434             }
1435         if result.returncode != 0:
1436             msg = str(
1437                 result.report.get("message")
1438                 or f"cloud compile job failed (returncode {result.returncode})."
1439             )
1440             return {"status": "failed", "message": msg, "video_id": video_id}
1441
1442         download_url = str(
1443             result.report.get("download_url")
1444             or f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
1445         )
1446         logger.info(
1447             "[COMPILE] cloud_run stitched: execution=%s out=%s video_id=%s rendition=%s",
1448             result.execution,
1449             result.outputs_uri,
1450             video_id,
1451             rendition,
1452         )
1453         notify("finished")
1454         return {
1455             "status": "success",
1456             "filepath": str(result.report.get("reel_uri") or result.outputs_uri or ""),
1457             "download_url": download_url,
1458             "video_id": video_id,
1459             "reel_uri": str(result.report.get("reel_uri") or ""),
1460             "filename": str(result.report.get("filename") or out_name),
1461             "rendition": str(result.report.get("rendition") or rendition),
1462             "compute": {
1463                 "path": "cloud_run",
1464                 "execution": result.execution,
1465                 "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
1466                 "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
1467             },
1468         }
1469
1470
1471     def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
1472         """Handler for a ``kind='compile'`` job (#329): re-resolve the segments, stitch the
reel, and return
1473         the ``{status, filepath, download_url}`` result the SPA polls for. Returns a
``status='failed'``
1474         result for an EXPECTED problem (missing video / no segments / a stitch error) so the
SPA always gets
1475         a clean verdict instead of a worker-crash error.

```

```

1476
1477 #521 phase_placement.compile: when compile is placed on the L4 (`cloud_run_l4`)
and cloud
1478 dispatch is available, the CPU-bound 4K stitch is OFFLOADED to the GPU/NVENC worker
1479 (`_dispatch_compile_remote`) ? the 2026-07-08 CUJ measured ~13 min for an 11-clip
reel on the
1480 2-vCPU control-plane. DEFAULT (compile on `control_plane`) stitches in-process
here, unchanged.""
1481 notify = progress or (lambda stage: None)
1482 params = json.loads(job.get("params") or "{}")
1483 video_id = job.get("video_id") or ""
1484 segment_ids = params.get("segment_ids") or []
1485 locale = params.get("locale")
1486 output_filename = params.get("output_filename", "")
1487 rendition = str(params.get("rendition") or "original")
1488 if rendition not in {"original", "sd"}:
1489     return {
1490         "status": "failed",
1491         "message": f"Unsupported compile rendition: {rendition}",
1492         "video_id": video_id,
1493     }
1494
1495 if (
1496     resolve_phase_placement(cfg, PHASE_COMPILE) == PLACE_CLOUD_L4
1497     and _cloud_available(cfg)
1498 ):
1499     return _dispatch_compile_remote(
1500         cfg, db, video_id, segment_ids, output_filename, locale, notify, rendition
1501     )
1502
1503 notify("resolving")
1504 try:
1505     video, kept, out_name = _resolve_compile(
1506         db, cfg, video_id, segment_ids, output_filename
1507     )
1508 except _CompileError as e:
1509     return {"status": "failed", "message": e.detail, "video_id": video_id}
1510
1511 stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1512 time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
1513 return _stitch_reel(
1514     cfg,
1515     stitching,
1516     video_id,
1517     video["filepath"],
1518     time_pairs,
1519     out_name,
1520     locale,
1521     notify,
1522     rendition,
1523 )
1524

```

7. user (2026-07-08T06:17:39.702Z)

```

backend\main.py:246: signed_reel_url,
backend\main.py:1046: def download_highlight(video_id: str, filename: str, request: Request):
backend\main.py:1058: # exists/sign logic (with the active storage config) lives in
orchestration.signed_reel_url to keep
backend\main.py:1061: signed = signed_reel_url(request.app.state.config, video_id, name)
backend\main.py:1080: detail=f"No compiled file named '{name}'. Compile first via
/api/compile.",
backend\orchestration.py:1180: def signed_reel_url(cfg, video_id: str, name: str) ->
Optional[str]:

```

8. user (2026-07-08T06:17:42.834Z)

```
1046 def download_highlight(video_id: str, filename: str, request: Request):
1047     """Stream a compiled highlight reel ? `filename` is the bare name given to
/api/compile
1048     (which only writes into output_dir; the browser previously got back an unreachable
1049     server-local path)."""
1050     try:
1051         name = safe_output_filename(filename)
1052     except ValueError as e:
1053         raise HTTPException(status_code=400, detail=str(e)) from e
1054     if not request.app.state.db.get_video(video_id):
1055         raise HTTPException(status_code=404, detail="Indexed video record not found.")
1056     # #463: when the reel was mirrored to the output bucket, 307 to a short-lived signed
URL (bytes flow
1057     # client<->store, bypassing Cloud Run's ~32 MiB response cap + the ephemeral local
/tmp). The
1058     # exists/sign logic (with the active storage config) lives in
orchestration.signed_reel_url to keep
1059     # this module lean; best-effort ? any error falls back to the local file below.
1060     try:
1061         signed = signed_reel_url(request.app.state.config, video_id, name)
1062         if signed:
1063             return RedirectResponse(signed, status_code=307)
1064     except Exception as e: # noqa: BLE001 ? signing/exists hiccup ? serve the local
file instead
1065         logger.warning("[highlights] signed-url path failed for %s; serving local: %s",
video_id, e)
1066     output_dir = (
1067         getattr(
1068             getattr(request.app.state.config, "output", None), "output_dir", "output"
1069         )
1070         or "output"
1071     )
1072     path = os.path.join(output_dir, name)
1073     try:
1074         path = resolve_under_root(path, output_dir)
1075     except ValueError as e:
1076         raise HTTPException(status_code=400, detail=str(e)) from e
1077     if not os.path.isfile(path):
1078         raise HTTPException(
1079             status_code=404,
1080             detail=f"No compiled file named '{name}'. Compile first via /api/compile.",
1081         )
1082     media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
1083     return FileResponse(path, media_type=media, filename=name)
1084
1085
1086 @app.post("/api/query")
1087 def query_play_segments(req: QueryRequest, request: Request):
1088     filters = {
1089         "server": req.server,
1090         "receiver": req.receiver,
```

9. assistant (2026-07-08T06:18:46.330Z)

I've traced the full chain. Let me verify one remaining link ? that the dispatcher does not re-check bucket existence and simply trusts the worker's returncode ? which I already read at lines 1435-1468. I have enough to render a verdict.

Let me confirm my understanding of the marker/put separation by re-reading the relevant worker lines is complete. I have all the evidence.

Analysis of the claim:

The claim describes a chain. I verified each link in the actual code:

1. **``_stitch_reel` swallows the mirror-upload failure and returns success** ?
`backend/orchestration.py:1287-1309`. After stitching to a scratch `out\_path`, `reel\_uri = reel\_object\_uri(...)`; `if reel\_uri:` it calls `storage.put(out\_path, reel\_uri, ...)` inside a `try/except` that on any exception only does `logger.warning("[compile] could not mirror reel to %s: %s", ...)` (line 1294-1295) and falls through to `return {"status": "success", ..., "reel\_uri": reel\_uri or ""}` (line 1301-1309). The comment at 1282-1286 explicitly states "On the L4 worker the mirror is how the reel reaches the bucket at all (the worker's local file is discarded when the Job exits)" ? yet the swallow-and-succeed is unconditional for both the CP and worker callers.
2. **Worker reports rc=0 / status=success** ? `backend/worker/\_\_main\_\_.py:506-524`.
`cmd\_compile` calls the shared `\_stitch\_reel`, then `status = str(result.get("status") or "failed")`, `rc = 0 if status == "success" else 3`, and writes a compile marker carrying `status='success' + the `download_url`.
3. **Dispatcher trusts rc=0, never re-verifies the reel is actually in the bucket** ?
`backend/orchestration.py:1435-1468`. It only checks `if result.returncode != 0:`; on rc=0 it returns `{"status": "success", "download\_url": ...}`. No `storage.exists(reel\_uri)` re-check.
4. **Download 404s while the CP has no local copy to fall back to** ?
`backend/main.py:1046-1083` + `backend/orchestration.py:1180-1191`. `signed\_reel\_url` computes `reel\_uri`, and `storage.exists(reel\_uri)` returns False (mirror failed) ? returns None ? endpoint falls back to `os.path.join(output\_dir, name)` on the **control-plane**, where the worker-stitched file never existed ? HTTP 404 "No compiled file named ...".

Refutation attempts, all failed:

- A total-storage-outage would also fail the marker put (`write\_compile\_marker`, worker line 462-463 swallows it ? no marker ? dispatcher times out rc4/rc5, a correct failure). But the reel put and the marker put are **separate operations on different-sized objects** (a large 4K reel vs. a tiny JSON marker). A partial/transient failure where the large reel upload fails (timeout/size/quota/mid-transfer blip) while the tiny marker succeeds is entirely realistic and reachable ? that is exactly the false-success case.
- The worker's local file is on the Cloud Run Job's ephemeral scratch, a separate instance from the control-plane; there is no shared FS, confirming no CP fallback exists.
- The dispatcher does not re-check bucket existence, so nothing downstream catches the missing reel.

The cited line number (1263) is off from the current checkout (the mirror `except` is at 1294-1295 and the success return at 1301-1309), but the described logic is present verbatim and behaves exactly as claimed; the line drift is just a snapshot difference and does not refute the defect.

This is a genuine "silent data loss reported as success" bug on the L4 cloud-compile path, violating the invariant that a stitch/delivery failure must surface as a clean failed job.

10. user (2026-07-08T06:18:56.892Z)

Structured output provided successfully